DOCKER CTF

Vulnerability Assessment & Proof of Concept Report

Victim hostname: vlundocker.lan **Victim IP:** 192.168.31.113

Assessment scope: Authorized, isolated Docker CTF lab only. Other IP addresses and other discovered host/device names have been removed from this report. The victim hostname and victim IP are retained.

Evidence sources: Supplied screenshots, *report1(4).xlsx*, and *kioptrix(1).pdf* as a style/PoC reference. The Kioptrix target details are not reused. Where evidence does not prove a service belongs to the victim, the PoC below is marked as a validation step rather than a confirmed exploit.

Executive Summary

The strongest target-specific evidence concerns the WordPress application on TCP/8000: WPScan identifies WordPress 4.8.28, XML-RPC is enabled, the user **bob** is enumerated, and a screenshot records a successful XML-RPC authentication attempt with a weak password. A request to the Twenty Seventeen theme's 404.php path also produces a PHP fatal error with a filesystem path. A reverse-shell screenshot shows command execution as **www-data**. The spreadsheet separately claims FTP/vsftpd 2.3.4 on TCP/21, but the supplied screenshots do not confirm that service on the victim; it remains an unverified lead.

Target Details

Field	Value
Victim hostname	vlundocker.lan
Victim IP	192.168.31.113
Observed web application	http://192.168.31.113:8000/
Environment	Docker CTF lab

How to read the PoCs

Each PoC is written in the Kioptrix report style: objective, numbered steps, command or observed action, expected/observed result, and remediation. Commands are scoped to the victim IP. Ports seen only on other internal Docker-network hosts are not asserted as confirmed victim services; run the relevant validation command before classifying them as findings.

1. Initial reconnaissance and port inventory

Objective: Establish which TCP ports and service banners are actually exposed by the victim host.

Step 1: Run a target-specific service scan from the authorized Kali lab VM.

```
nmap -sV -sC -Pn -p- 192.168.31.113
```

Step 2: Record only ports returned for *192.168.31.113*. The screenshots show TCP/8000 as the WordPress URL. Other ports in the Docker-network port-scan screenshot were associated with internal network hosts and must be confirmed against this victim IP.

Evidence/interpretation: This scan creates the authoritative per-victim port list. Do not copy internal network ports into the final confirmed list unless this scan shows them open.

Remediation: Close unused ports and restrict management/database services to trusted lab networks.

2. WordPress version disclosure / outdated WordPress [High]

Evidence from supplied material: WPScan output identifies WordPress 4.8.28 on the web application at TCP/8000.

Proof of Concept (PoC)

Step 1: Fingerprint the application and enumerate versions using WPScan.

Step 2: Compare the detected core/theme versions against current vendor security advisories. Do not assign a specific CVE solely from the version string.

```
wpscan --url http://192.168.31.113:8000/ --enumerate u,v
```

Observed / expected result: The supplied WPScan screenshot reports WordPress 4.8.28 and the Twenty Seventeen theme. This is evidence of an outdated installation; exploitability of a particular CVE needs separate validation.

Remediation: Upgrade WordPress core and themes to supported releases, back up first, test changes, and rerun the scan.

3. XML-RPC endpoint enabled [Medium]

Evidence from supplied material: WPScan output explicitly reports XML-RPC as enabled at /xmlrpc.php.

Proof of Concept (PoC)

Step 1: Request the XML-RPC endpoint and inspect the HTTP response.

Step 2: If it is not needed by the site, disable it; otherwise restrict access and monitor authentication attempts.

```
curl -i http://192.168.31.113:8000/xmlrpc.php
```

Observed / expected result: The endpoint responds over HTTP if reachable. The screenshot already records WPScan's detection of XML-RPC. Endpoint availability alone is not proof of a standalone CVE.

Remediation: Disable XML-RPC if unused. If required, restrict access, enforce strong credentials/MFA where possible, and apply rate limiting.

4. WordPress user enumeration [Low]

Evidence from supplied material: The WPScan output identifies the WordPress user/author account bob.

Proof of Concept (PoC)

Step 1: Make a benign author-pattern request and inspect the response/redirect.

Step 2: Compare the result with the user enumeration shown in the supplied WPScan screenshot.

```
curl -i 'http://192.168.31.113:8000/?author=1'
```

Observed / expected result: A redirect or page metadata may disclose the author slug. The supplied WPScan screenshot separately identifies bob. This finding supports targeted-attack risk but is not direct account compromise.

Remediation: Reduce unnecessary public author enumeration where practical; use strong passwords, MFA, and login rate limiting.

5. Weak WordPress credentials (XML-RPC authentication) [High]

Evidence from supplied material: The supplied screenshot shows WPScan reporting a successful XML-RPC login for user bob with password Welcome1.

Proof of Concept (PoC)

Step 1: Review the supplied successful-authentication result as the PoC evidence; do not repeat password guessing against systems outside this isolated CTF.

Step 2: For remediation validation, change the account password and verify that the old credential no longer authenticates. Use a single controlled test, not a password spray.

```
# Evidence recorded in screenshot:
# Username: bob
# Password accepted in lab: Welcome1
# Remediation validation: rotate password, then test old credential once.
```

Observed / expected result: The screenshot reports a valid username/password combination. This is a confirmed weak-credential finding for the lab account.

Remediation: Immediately rotate the password, check WordPress and web-server logs for unauthorized access, enable MFA if available, and apply rate limits/lockout controls.

6. PHP fatal error and filesystem-path disclosure [Medium]

Evidence from supplied material: A screenshot of the Twenty Seventeen theme's 404.php URL displays a PHP fatal error and reveals a server filesystem path.

Proof of Concept (PoC)

Step 1: Request the path shown in the evidence from the victim web application.

Step 2: Record the HTTP status and body. Do not treat the error as proof of code execution; the screenshot proves error disclosure only.

Step 3: Retest after remediation and confirm the response no longer reveals internal paths or stack details.

```
curl -i http://192.168.31.113:8000/wp-content/themes/twentyseventeen/404.php
```

Observed / expected result: The supplied browser screenshot shows a PHP fatal error involving get_header() and a filesystem path. This is information disclosure/application misconfiguration.

Remediation: Disable display_errors in production, log errors server-side, and update/repair the theme. Avoid exposing raw PHP errors to remote users.

7. Reverse shell / command execution as www-data [High]

Evidence from supplied material: A supplied listener screenshot shows a connected shell with uid=33(www-data), gid=33(www-data), and the whoami result www-data.

Proof of Concept (PoC)

Step 1: Use the already supplied shell screenshot as proof that a shell session was obtained in the lab. The evidence does not include a complete, target-specific payload command, so no payload is invented in this report.

Step 2: In the established lab shell, verify the execution context with the following read-only commands.

Step 3: Record the output and confirm the identity is limited to the web-server account.

```
id
whoami
ip a
```

Observed / expected result: The screenshot confirms command execution as www-data inside the container. It does not establish root access or host-level compromise.

Scope caveat: The screenshot includes a listener and shell identity but does not establish which exact exploit/payload created the session.

Remediation: Identify and fix the initial web-application weakness, remove unauthorized shell access, rotate exposed credentials, review logs, rebuild the affected container from a trusted image, and run the container with least privilege.

8. FTP / vsftpd 2.3.4 on TCP/21 — spreadsheet lead [Critical if confirmed; unverified]

Evidence from supplied material: report1(4).xlsx lists FTP, port 21, service vsftpd, version 2.3.4, and Critical severity. The provided screenshots do not show this service on the victim IP.

Proof of Concept (PoC)

Step 1: Verify whether TCP/21 is open on the victim with a target-specific scan.

Step 2: If open, inspect the service banner and validate the exact package/build through authorized lab access. A banner alone can be misleading.

Step 3: Only classify the vulnerability as confirmed after the service/build is independently validated.

```
nmap -sV -Pn -p 21 192.168.31.113
```

Observed / expected result: No screenshot currently confirms FTP/21 on the victim. The spreadsheet entry is retained as an unverified lead rather than a proven vulnerability.

Scope caveat: No exploit execution is documented because the supplied evidence does not establish that this service exists on the victim.

Remediation: If confirmed vulnerable, isolate/disable the service and replace it with a trusted supported package. Re-scan after patching.

9. Additional ports observed in Docker network screenshots

The supplied port-scan screenshots show TCP/22, TCP/80, TCP/3306, TCP/8000, and TCP/8022 on internal Docker-network addresses. Because the instruction is to remove other IP addresses and names, those addresses are omitted here. These observations are not enough to claim every port belongs to 192.168.31.113. Use the per-port validation PoCs below against the victim IP; report a port as open only if the command confirms it.

Port / service to validate	PoC command	Interpretation
22 / SSH	nmap -sV -Pn -p 22 192.168.31.113	If open, record SSH banner/version; exposure alone is not proof of a vulnerability.
80 / HTTP	nmap -sV -Pn -p 80 192.168.31.113	If open, fingerprint the web service and review its content/headers.
111 / RPCBind (listed in the Kioptrix sample only)	nmap -sV -Pn -p 111 192.168.31.113	Not demonstrated for this victim in the supplied screenshots; validate before including.
443 / HTTPS (listed in the Kioptrix sample only)	nmap -sV -Pn -p 443 192.168.31.113	Not demonstrated for this victim in the supplied screenshots; validate before including.
1024 / RPC-related (listed in the Kioptrix sample only)	nmap -sV -Pn -p 1024 192.168.31.113	Not demonstrated for this victim in the supplied screenshots; validate before including.
3306 / MySQL	nmap -sV -Pn -p 3306 192.168.31.113	If open, confirm database identity and restrict network access.
8000 / WordPress web app	nmap -sV -Pn -p 8000 192.168.31.113	Target-specific web service evidenced by supplied URL and WPScan output.
8022 / forwarded SSH-like service	nmap -sV -Pn -p 8022 192.168.31.113	Seen in port-forwarding screenshot; confirm ownership/reachability on victim.

Port inventory convenience command

```
nmap -sV -sC -Pn -p 21,22,80,111,443,1024,3306,8000,8022 192.168.31.113
```

This command is a validation inventory, not a claim that all listed ports are open. The Kioptrix report was used for presentation style only; Kioptrix-specific Samba, RPC, and SSL findings are not attributed to the Docker target without evidence.

10. Consolidated remediation plan

Priority	Action
1 — Immediate	Change the weak WordPress password and review authentication/access logs.
2 — High	Upgrade WordPress core and theme to supported versions; disable public PHP error display.
3 — High	Investigate the source of the www-data shell and rebuild/restore the container after remediation.
4 — High if confirmed	Validate the spreadsheet's vsftpd 2.3.4 claim on the victim; isolate/replace the package if confirmed.
5 — Medium	Disable/restrict XML-RPC if not required; apply login rate limiting and MFA where available.
6 — Medium	Confirm ownership of internal SSH, HTTP, MySQL, and forwarded ports; close or firewall unnecessary services.

Limitations and evidence integrity

This report does not fabricate screenshots or exploit output. Where the supplied evidence supports an observed result, that result is described directly. Where a port/service appears only on an internal Docker-network host or in the spreadsheet, the corresponding PoC is a validation step and the report clearly labels the finding as unconfirmed. No unrelated discovered IPs or host/device names are reproduced.

All testing commands are intended only for the authorized, isolated Docker CTF target.